

ASTra: Edge-Native Security Action Layer

Team: Root Coders

Lead Developer: Madhu S

Event: iQOO City Battle Hackathon (Aug–Oct 2026)

Track: 06 - Developer Tools

Format: Phone-First (Red Light / Green Light)

1. Executive Summary

As the software industry rapidly adopts autonomous AI coding agents (such as GitHub Copilot, AutoGPT, and open-source LLMs), a critical security vulnerability has emerged:

Unsafe execution.

If an AI hallucinates or misinterprets a prompt, it can execute terminal commands that destroy local databases, leak Intellectual Property, or wipe the user's hard drive (`rm -rf /`). Sending code to cloud models for security validation is slow, costly, and fundamentally compromises code privacy.

ASTra solves this. We have built an execution firewall that decouples code generation from code execution. It grades AI intents into 3 Risk Tiers *before* they ever touch the terminal. Instead of relying on the cloud, ASTra places the security evaluation natively on the **iQOO smartphone**, creating an Edge-Native Security Hardware Key.

2. The Problem Statement (Deep Dive)

Developers want AI agents to automate tedious tasks: writing boilerplate, setting up environments, and executing tests. However, giving an AI unrestricted access to the host machine's terminal is extremely dangerous.

- **Context Blindness:** Feeding an entire 2,000-line codebase to an AI just to review a 3-line change overwhelms the model's context window, leading to bad code execution.
- **The Cloud Bottleneck:** Pushing enterprise source code to cloud APIs (like OpenAI) for safety reviews violates data privacy policies and introduces massive latency.
- **The "Babysitting" Problem:** Because developers do not trust AI agents to run commands autonomously, they spend more time babysitting the AI than actually writing code, defeating the purpose of automation.

3. The ASTra Solution & "Phone-First" Architecture

ASTra bridges the gap between developer productivity and enterprise security by turning the smartphone into a zero-trust hardware key.

Our system flawlessly integrates the iQOO hackathon constraints:

1. **The Laptop (Green Light):** The developer works on their laptop. When the AI agent attempts to execute a shell command or push a Git commit, the local ASTra terminal intercepts and pauses the execution.
 2. **Office Kit Bridge:** The intercepted command, along with the relevant code snippet, is beamed seamlessly to the iQOO device via the iQOO Office Kit framework.
 3. **Snapdragon NPU Validation:** The iQOO phone acts as the physical security brain. A local, open-source LLM runs natively on the Snapdragon NPU, evaluating the threat level completely offline.
-

4. Deep Technical Infrastructure

ASTra is not a basic API wrapper. It is a deep, underlying infrastructure optimized specifically for edge devices:

- **Surgical AST Slicing (Tree-Sitter):** LLMs running locally on a phone have very small memory contexts. We implemented C-based Tree-sitter parsing to extract *only the exact method or class* that changed (e.g., slicing 10 lines out of a 2,000-line file). This surgical extraction prevents the Snapdragon NPU from crashing under heavy memory loads and ensures hyper-fast inference.
 - **Local Offline Memory (ChromaDB):** We built a local RAG (Retrieval-Augmented Generation) semantic indexing system. ASTra indexes the project files into a local SQLite/Chroma database so the AI understands the codebase without needing internet access.
 - **Ephemeral Sandboxing (Docker):** Safe commands execute instantly inside an isolated, disposable Docker container on the host machine, guaranteeing that even malicious code cannot contaminate the host OS.
 - **CI/CD Interception (FastAPI):** We built an automated Webhook server that intercepts GitHub Pull Requests in real-time for continuous security scanning.
-

5. The 3-Tier Security Framework (Policy Gate)

ASTra dynamically grades intents into 3 tiers:

-  **Tier 1 (Safe):** Read-only commands (e.g., `echo`, `ls`, `git status`). Executed automatically and instantly.
-  **Tier 2 (Write):** Non-destructive writes (e.g., `touch test.txt`, `pytest`). Executed automatically, but *strictly routed inside the Docker sandbox*.
-  **Tier 3 (Destructive):** High-risk actions (e.g., `rm -rf`, `git push --force`, `drop database`).

The iQOO Hardware Key Interaction

When a Tier 3 command is detected, it is hard-blocked on the laptop. A physical alert triggers on the iQOO smartphone screen. The developer must pick up the phone and use **Voice Commands ("Approve Execution")** into the phone's microphone, or use FaceID biometric authentication, to authorize the dangerous deployment.

6. Hackathon Rubric Alignment

ASTra is engineered to perfectly hit all 6 dimensions of the iQOO City Battle scoring rubric:

1. **End Product Quality (30%):** A fully functioning, 11-commit Python prototype that actually intercepts commands and spins up real Docker containers.
2. **Novelty & Impact (20%):** Solves a massive enterprise security problem (AI Safety) rather than building another consumer chatbot.
3. **Creative Phone Use (15%):** Using the iQOO phone as a physical "Hardware Security Key" with Voice/Biometric approvals for Tier 3 overrides.
4. **Technical Depth (15%):** Implementation of abstract syntax trees (Tree-sitter), Vector Databases (ChromaDB), and containerized execution (Docker).
5. **Office Kit Usage (10%):** Real-time laptop-to-phone orchestration for command interception.
6. **Demo & Presentation (10%):** A flawless Red Light / Green Light demonstration of a blocked destructive command.

7. Future Vision

ASTra proves that the future of enterprise security belongs on the edge. By heavily utilizing the Snapdragon NPU, Office Kit, and the physical sensors of the iQOO device, we have transformed a consumer smartphone into a necessary piece of enterprise developer hardware. In Phase 2, we plan to release ASTra as an open-source VSCode extension natively integrated with local LLMs.